# Bash Scripting in Linux - Student Lab Workbook

## Create Your First Bash Script

1. Create myscript.sh
2. Add #!/bin/bash in the first line.
3. Print Hello World
4. Run chmod +x and execute.

```
#!/bin/bash
echo "Hello World"
```

## Using Variables

5. Create variables for your name and course.
6. Display both values using echo.

```
#!/bin/bash

name="Andrew"
course="Network Operating Systems Security"
echo $name
echo $course
```

## User Input

7. Prompt the user for their name and display a greeting.

```
#!/bin/bash
read -p "Enter your name: " name
echo "Hello $name"
```

## Arithmetic Operations

8. Create two numeric variables.
9. Calculate addition, subtraction and multiplication.

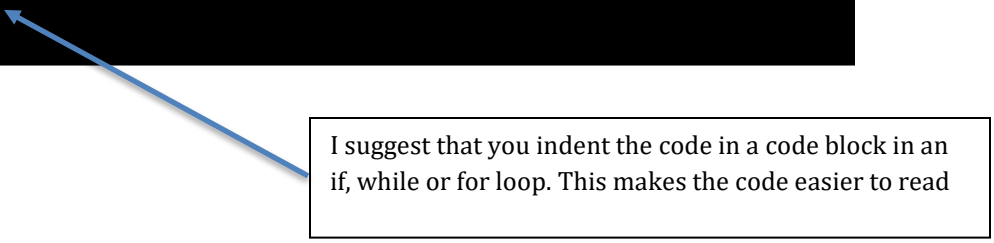
```
#!/bin/bash
a=10
b=5
echo $((a+b))
echo $((a-b))
echo $((a*b))
```

10. In challenge, below, I ask can you make a calculator. Do you think that you can make the operator a variable

## If Statements

11. Check whether a user is 18 years or older.

```
#!/bin/bash
if [ $age -ge 18 ]
then
   echo "Adult"
fi
```



## If Else Statements

12. Check if a file exists.

```
#!/bin/bash
if [ -f test.txt ]
then
   echo "Exists"
else
   echo "Missing"
fi
```

## While Loops

13. Display numbers 1 to 5.

```
#!/bin/bash
count=1                    # Initialise the condition
while [ $count -le 5 ]     # Check the condition
do
   echo $count
   ((count++))             # Update the condition
done
```

## For Loops

14. Display all text files in the current directory.

```
#!/bin/bash
for file in *.txt
do
 echo $file
done
```

## Command Line Arguments

15. Pass a name to a script using $1.

```
#!/bin/bash
echo "Hello $1"
```

## Functions

16. Create a reusable function that accepts a name.

```
#!/bin/bash
greet(){
   echo "Hello $1"
}
greet Student
```

## Challenge Tasks

1. Put the command you used to count the number of failed attempts in the honey pot logs into a script
2. Create a calculator that accepts two numbers and an operation.
3. Read usernames from a file and display a message for each user.
4. Write a script that has an infinite loop that keeps counting up. It should output the value of the first argument every 1000 in the count.

   Run that script twice in the background with different priorities (and a different argument), both outputting to the same file.

   See which one has the most output.

   ```
   nice -10 script.sh low & > output.txt
   nice -1 script.sh high &> output.txt
   cat output.txt
   ```

   Kill them